

AI/ML Spam Message Detector — Full Project Prompt

Continue my AI/ML Spam Message Detector project from the complete context of this chat and help me finish it as a working real-time Telegram application.

PROJECT BACKGROUND:

I built an AI/ML Spam Message Detector using the SMS Spam Collection dataset. In Google Colab, I uploaded the dataset, inspected the data, converted the labels (ham = 0, spam = 1), trained the machine-learning model, tested it, and achieved approximately 96.23% accuracy. I also tested custom messages and confirmed that the model can classify messages as Spam or Ham.

CURRENT PROJECT FILES:

My VS Code project contains:

- bot.py
- spam_model.pkl
- vectorizer.pkl
- requirements.txt
- README.md
- render.yaml

The trained model and TF-IDF vectorizer were saved as pickle files and are loaded by bot.py.

TELEGRAM BOT:

I created a Telegram bot and connected it to bot.py. The bot works locally. When a user sends a message, the bot uses the trained model and vectorizer to classify it and replies with either Spam or Ham (Not Spam).

The bot code uses python-telegram-bot and contains the trained model, vectorizer, Telegram token, message handler, prediction logic, and polling/startup code.

GITHUB:

The project has already been initialized with Git, committed, and pushed to GitHub.

Repository: <https://github.com/surya06143/spam-telegram-bot>

CURRENT HOSTING STAGE:

I attempted to deploy the Telegram bot using Render. I created a Background Worker and entered:

Build Command: `pip install -r requirements.txt`

Start Command: `python bot.py`

However, Render showed paid instance options, so I need a genuinely free hosting/deployment solution if one is available. If the chosen service has limitations, clearly explain them before proceeding.

IMPORTANT REQUIREMENTS:

1. Do not restart the entire project from the beginning.
2. Continue from the current state described above.
3. Keep my existing files and code whenever possible.
4. Only modify code when necessary for deployment, compatibility, security, or reliability.
5. Give instructions step-by-step, one stage at a time.
6. Assume I am a beginner and explain exactly what to click, what command to type, and what output I should expect.
7. Do not give multiple confusing alternatives unless the first method fails.
8. Help me deploy the bot so it can run continuously without keeping my laptop or VS Code open, preferably using a free service.
9. Help me test the deployed Telegram bot from Telegram.
10. If there is a deployment error, ask me to send a screenshot or error message and then diagnose it.
11. Make sure the deployment includes all required files, especially bot.py, requirements.txt, spam_model.pkl, and vectorizer.pkl.

12. Check that the Python version and package versions are compatible with the deployment platform.
13. Do not expose or repeat my Telegram bot token in the final instructions. Treat the token as secret. If it was previously exposed, explain that I should regenerate it through BotFather and update the deployment securely using an environment variable.
14. For production deployment, prefer storing the Telegram token in an environment variable rather than directly inside bot.py.
15. Explain how to update the GitHub repository after any required code changes.
16. Explain how to confirm that the deployed worker is actually running.
17. Explain how to test several normal and spam-like messages safely and verify the predictions.
18. At the end, provide a short checklist confirming that the project is ready to demonstrate as a real-time AI/ML Telegram Spam Message Detector.

EXPECTED FINAL PROJECT:

User sends a message to the Telegram bot → Telegram delivers the message → deployed Python bot receives it → TF-IDF vectorizer converts the text into features → trained ML model predicts Spam/Ham → bot immediately replies with the classification.

Please continue from the current deployment stage and guide me through the remaining steps until the Telegram bot is successfully running online.